

Lexical Analysis(NLP)

Text Normalization

A set of tasks which collectively converts a text to a more convenient, standard form is called text normalization.

Tokenization: separating out or tokenizing words from running text

- **Lemmatization:** the task of determining that two words have the same root, despite their surface differences
- **Stemming** refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word.
- **Sentence segmentation:** breaking up a text into individual sentences, using cues like periods or exclamation points.

Regular Expressions

- One of the most useful tools for text processing in computer science has been the **regular expression** (often shortened to **regex**).
- It is a language for specifying text search strings.
- Formally, a regular expression is an algebraic notation for characterizing a set of strings.
- A regular expression search function will search through the corpus (a single document or a collection) , returning all texts that match the pattern.
- Regular expressions come in many variants. We'll be describing extended regular expressions

Regular expressions are used everywhere

- Part of every text processing task
 - Not a general NLP solution
 - But very useful as part of those systems (e.g., for pre-processing or text formatting)
- Necessary for data analysis of text data
- A widely used tool in industry and academics

Basic Regular Expression Patterns

If you like to search a string, let's say "Bunny", you can do that by `/Bunny/`.

Regular expressions are case sensitive; lower case `/s/` is distinct from upper case `/S/` (`/s/` matches a lower case `s` but not an upper case `S`).

This means that the pattern `/woodchucks/` will not match the string `Woodchucks`.

We can solve this problem with the use of the square braces `[and ]`.

The string of characters inside the braces specifies a **disjunction** of characters to match.

Regex	Example Patterns Matched
/woodchucks/	“interesting links to <u>woodchucks</u> and lemurs”
/a/	“Ma <u>r</u> y Ann stopped by Mona’s”
/!/	“You’ve left the burglar behind again <u>!</u> ” said Nori

Regex	Match	Example Patterns
/[wW]oodchuck/	Woodchuck or woodchuck	“ <u>Woodchuck</u> ”
/[abc]/	‘a’, ‘b’, or ‘c’	“In uo <u>m</u> ini, in solda <u>t</u> i”
/[1234567890]/	any digit	“plenty of <u>7</u> to 5”

Regular Expressions: Ranges

- The regular expression `/[1234567890]/` specifies any single digit.
- The regular expression `/[ABCDEFGHIJKLMNOPQRSTUVWXYZ]/` matches with any capital letter.
- In cases where there is a well-defined sequence associated with a set of characters, the brackets can be used with the dash (-) to specify any one character in a range.

Ranges using the dash:

Pattern	Matches	
<code>[A-Z]</code>	An upper case letter	<u>D</u> renched Blossoms
<code>[a-z]</code>	A lower case letter	<u>m</u> y beans were impatient
<code>[0-9]</code>	A single digit	Chapter <u>1</u> : Down the Rabbit Hole

Regular Expressions: Negation in Disjunction

Carat as first character in [] negates the list

- Note: Carat means negation only when it's first in []
- Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
[^A-Z]	Not an upper case letter	O <u>y</u> fn pripetchik
[^Ss]	Neither 'S' nor 's'	<u>I</u> have no exquisite reason"
[^.]	Not a period	<u>O</u> ur resident Djinn
[e^]	Either e or ^	Look up <u>^</u> now

Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
<code>\d</code>	<code>[0-9]</code>	Any digit	Fahreheit <u>4</u> 51
<code>\D</code>	<code>[^0-9]</code>	Any non-digit	<u>B</u> lue Moon
<code>\w</code>	<code>[a-zA-Z0-9_]</code>	Any alphanumeric or <code>_</code>	<u>D</u> aiyu
<code>\W</code>	<code>[^\w]</code>	Not alphanumeric or <code>_</code>	Look <u>!</u>
<code>\s</code>	<code>[\r\t\n\f]</code>	Whitespace (space, tab)	Look <u>_</u> up
<code>\S</code>	<code>[^\s]</code>	Not whitespace	<u>L</u> ook up

Regular Expressions: More Disjunction

Groundhog is another name for woodchuck!

The pipe symbol | for disjunction

Pattern	Matches
<code>groundhog woodchuck</code>	woodchuck
<code>yours mine</code>	yours
<code>a b c</code>	= <code>[abc]</code>
<code>[gG]roundhog [Ww]oodchuck</code>	Woodchuck



Wildcards, optionality, repetition: . ? * +

Pattern	Matches	Examples
<code>beg.n</code>	Any char	<u>begin</u> <u>begun</u> <u>beg3n</u> <u>beg n</u>
<code>woodchucks?</code>	Optional s	<u>woodchuck</u> <u>woodchucks</u>
<code>to*</code>	0 or more of previous char	<u>t</u> <u>to</u> <u>too</u> <u>tooo</u>
<code>to+</code>	1 or more of previous char	<u>to</u> <u>too</u> <u>tooo</u> <u>toooo</u>



Stephen C Kleene

Kleene *, Kleene +

Use of curly Brace

In regular expressions, the curly braces `{}` are used as a quantifier to specify the number of times the preceding element (character, group, or character class) must appear in the text to be considered a match.

There are three main forms of using `{}` for repetition:

Exact Number of Times:

`{n}`: Matches the preceding element exactly `n` times.

Example: `a{3}` matches "aaa".

At Least a Minimum Number of Times:

`{n,}`: Matches the preceding element at least `n` times, with no upper limit on the number of repetitions.

Example: `b{2,}` matches "bb", "bbb", "bbbb", and so on.

Within a Range of Times:

`{n,m}`: Matches the preceding element between `n` (inclusive) and `m` (inclusive) times.

Example: `c{1,3}` matches "c", "cc", or "ccc".

Regular Expressions: Anchors

^ **\$**

Pattern	Matches
<code>/^The/</code>	Matches a line starts with "The"
<code>^[A-Z]</code>	<u>P</u> alo Alto
<code>^[^A-Za-z]</code>	<u>1</u> "Hello"
<code>\.\$</code>	The end <u>.</u>
<code>.\$</code>	The end <u>?</u> The end <u>!</u>
<code>/^The dog\.\$/</code>	Matches a line that contains only the phrase "The dog."

	Regex	Match
	<code>^</code>	start of line
	<code>\$</code>	end of line
	<code>\b</code>	word boundary
	<code>\B</code>	non-word boundary

`\b` matches a word boundary, and `\B` matches a non word-boundary. Thus, `^bthe\b/` matches the word “the” but not the word “other”.

precedence

- How can we specify both guppy and guppies? We cannot simply say `/guppy|ies/`, because that would match only the strings guppy and ies.
- This is because sequences like guppy take precedence over the disjunction operator `|`.
- To make the disjunction operator apply only to a specific pattern, we need to use the parenthesis operators `(` and `)`.
- So the pattern `/gupp(y|ies)/` would specify that we meant the disjunction only to apply to the suffixes y and ies.

- Kleene* operator applies by default only to a single character, not to a whole sequence.
- Suppose we want to match repeated instances of a string.
- Perhaps we have a line that has column labels of the form Column 1
Column 2 Column 3.
- The expression `/Column_[0-9]+_*/` will not match any number of columns; instead, it will match a single column followed by any number of spaces!
- With the parentheses, we could write the expression `/(Column_[0-9]+_*)*/` to match the word Column, followed by a number and optional spaces, the whole pattern repeated zero or more times.

This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the operator precedence hierarchy for regular expressions.

Parenthesis	()
Counters	* + ? {}
Sequences and anchors	the ^my end\$
Disjunction	

The iterative process of writing regex's

Find me all instances of the word “the” in a text.

the

Misses capitalized examples

[tT]he

Incorrectly returns other or Theology

\W[tT]he\W

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

1. Not matching things that we should have matched
(The)

False negatives

2. Matching strings that we should not have matched
(there, then, other)

False positives

Characterizing work on NLP

In NLP we are always dealing with these kinds of errors.

Reducing the error rate for an application often involves two antagonistic efforts:

- Increasing coverage (or *recall*) (minimizing false negatives).
- Increasing accuracy (or *precision*) (minimizing false positives)

Regular expressions play a surprisingly large role

Widely used in both academics and industry

1. Part of most text processing tasks, even for big neural language model pipelines
 - including text formatting and pre-processing
2. Very useful for data analysis of any text data

Basic Text Processing

More Regular Expressions: Substitutions and ELIZA

Substitutions

Substitution in Python and UNIX commands:

```
s/regexp1/pattern/
```

e.g.:

```
s/colour/color/
```

Capture Groups

- Say we want to put angles around all numbers:
the 35 boxes → *the <35> boxes*
- Use parenthesis () to "capture" a pattern into a numbered register (1, 2, 3...)
- Use `\1` to refer to the contents of the register
`s / ( [0-9] + ) / <\1> /`

Capture groups: multiple registers

```
/the (.*?)er they (.*), the \1er we \2/
```

Matches

the faster they ran, the faster we ran

But not

the faster they ran, the faster we ate

But suppose we don't want to capture?

Parentheses have a double function: grouping terms, and capturing

Non-capturing groups: add a `?:` after parenthesis:

```
/(?:some|a few) (people|cats) like some \1/
```

matches

- some cats like some cats

but not

- some cats like some some

Lookahead assertions

`(?= pattern)` is true if pattern matches, but is **zero-width; doesn't advance character pointer**

`(?! pattern)` true if a pattern does not match

How to match, at the beginning of a line, any single word that doesn't start with "Volcano":

```
/^(?!Volcano) [A-Za-z]+/
```

Simple Application: ELIZA

Early NLP system that imitated a Rogerian psychotherapist

- Joseph Weizenbaum, 1966.

Uses pattern matching to match, e.g.,:

- "I need X"

and translates them into, e.g.

- "What would it mean to you if you got X?"

Simple Application: ELIZA

Men are all alike.

IN WHAT WAY

They're always bugging us about something or other.

CAN YOU THINK OF A SPECIFIC EXAMPLE

Well, my boyfriend made me come here.

YOUR BOYFRIEND MADE YOU COME HERE

He says I'm depressed much of the time.

I AM SORRY TO HEAR YOU ARE DEPRESSED

How ELIZA works

s/. * I'M (depressed|sad) . */I AM SORRY TO HEAR YOU ARE \1/
s/. * I AM (depressed|sad) . */WHY DO YOU THINK YOU ARE \1/
s/. * all . */IN WHAT WAY?/
s/. * always . */CAN YOU THINK OF A SPECIFIC EXAMPLE?/

Example

- I. Write a regex to match **integer literals** (positive and negative numbers).
- II. Write a regex to match an **email address**.
- III. Write a regex that matches any **5-letter English word**.
- IV. Write a regex to match a **date** in the format dd/mm/yyyy.

- I. Write a regex to match **integer literals** (positive and negative numbers).

`^-?\d+$`

- II. Write a regex to match an email address.

`^[\w\.-]+@[\w\.-]+\.\w+$`

- III. Write a regex that matches any **5-letter English word**.

`^[A-Za-z]{5}$`

- IV. Write a regex to match a **date** in the format dd/mm/yyyy.

`^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$`

Use regex to extract the required patterns from the given text:

Text:

My email is alice123@example.com.

I was born on 05/07/1998.

My friend's email is bob_smith@uni.edu and his number is +91-9876543210.

- I. Extract all email addresses
- II. Extract the date
- III. Extract the phone number

- I. `[\w\.-]+@[\w\.-]+\.\w+` Matches:
alice123@example.com
bob_smith@uni.edu
- II. `\d{2}/\d{2}/\d{4}` Matches:
05/07/1998
- III. `\+?\d{1,3}-\d{10}` Matches:
+91-9876543210

How many words in a sentence?

"I do uh main- mainly business data processing"

- Fragments, filled pauses

"Seuss's **cat** in the hat is different from other **cats**!"

- **Lemma**: same stem, part of speech, rough word sense
 - **cat** and **cats** = same lemma
- **Wordform**: the full inflected surface form
 - **cat** and **cats** = different wordforms

How many words in a sentence?

they lay back on the San Francisco grass and looked at the stars
and their

Type: an element of the vocabulary.

Token: an instance of that type in running text.

How many?

- 15 tokens (or 14)
- 13 types (or 12) (or 11?)

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, **$|V|$** is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^\beta$ where often $0.67 < \beta < 0.75$

i.e., vocabulary size grows with $>$ square root of the number of word tokens

	Tokens = N	Types = $ V $
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13+ million

Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer(s),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

Corpora vary along dimension like

- **Language:** 7097 languages in the world
- **Variety**, like African American English varieties.
 - AAE Twitter posts might include forms like "*iont*" (*I don't*)
- **Code switching**, e.g., Spanish/English, Hindi/English:
 - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:]
 - H/E: dost tha or ra- hega ... dont worry ... but dherya rakhe
["he was and will remain a friend ... don't worry ... but have faith"]
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics:** writer's age, gender, ethnicity, SES

Corpus datasheets

Gebbru et al (2020), Bender and Friedman (2018)

Motivation:

- Why was the corpus collected?
- By whom?
- Who funded it?

Situation: In what situation was the text written?

Collection process: If it is a subsample how was it sampled? Was there consent? Pre-processing?

+Annotation process, language variety, demographics, etc.

Basic Text Processing

Word tokenization

Text Normalization

Every NLP task requires text normalization:

1. Tokenizing (segmenting) words
2. Normalizing word formats
3. Segmenting sentences

Space-based tokenization

A very simple way to tokenize

- For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
- Segment off a token between instances of spaces

Unix tools for space-based tokenization

- The "tr" command
- Given a text file, output the word tokens and their frequencies

Simple Tokenization in UNIX

Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < shakes.txt
```

Change all non-alpha to newlines

```
| sort
```

Sort in alphabetical order

```
| uniq -c
```

Merge and count each type

```
1945 A
```

```
72 AARON
```

```
19 ABBESS
```

```
5 ABBOT
```

```
... ..
```

```
25 Aaron
```

```
6 Abate
```

```
1 Abates
```

```
5 Abbess
```

```
6 Abbey
```

```
3 Abbot
```

```
.... ..
```

The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

THE

SONNETS

by

William

Shakespeare

From

fairest

creatures

We

...

The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...

More counting

Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'a-z' '\n' | sort | uniq -c
```

Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
```

Issues in Tokenization

Can't just blindly remove punctuation:

- m.p.h., Ph.D., AT&T, cap'n
- prices (\$45.55)
- dates (01/02/06)
- URLs (<http://www.stanford.edu>)
- hashtags (#nlproc)
- email addresses (someone@cs.colorado.edu)

Clitic: a word that doesn't stand on its own

- "are" in [we're](#), French "je" in [j'ai](#), "le" in [l'honneur](#)

When should multiword expressions (MWE) be words?

- [New York](#), rock 'n' roll

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...      ([A-Z]\.)+        # abbreviations, e.g. U.S.A.
...      | \w+(-\w+)*      # words with optional internal hyphens
...      | \$?\d+(\.\d+)?%? # currency and percentages, e.g. $12.40, 82%
...      | \.\.\.          # ellipsis
...      | [][.,;"'()?:-_'] # these are separate tokens; includes ], [
...      '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Tokenization in languages without spaces

Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

How do we decide where the token boundaries should be?

Word tokenization in Chinese

Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")

Each one represents a meaning unit called a morpheme.

Each word has on average 2.4 of them.

But deciding what counts as a word is complex and not agreed upon.

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

5 words?

姚 明 进入 总 决赛

Yao Ming reaches overall finals

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

5 words?

姚 明 进入 总 决赛

Yao Ming reaches overall finals

7 characters? (don't use words at all):

姚 明 进 入 总 决 赛

Yao Ming enter enter overall decision game

Word tokenization / segmentation

So in Chinese it's common to just treat each character (zi) as a token.

- So the **segmentation** step is very simple

In other languages (like Thai and Japanese), more complex word segmentation is required.

- The standard algorithms are neural sequence models trained by supervised machine learning.

Basic Text Processing

Byte Pair Encoding

Another option for text tokenization

Instead of

- white-space segmentation
- single-character segmentation

Use the data to tell us how to tokenize.

Subword tokenization (because tokens can be parts of words as well as whole words)

Subword tokenization

Three common algorithms:

- **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
- **Unigram language modeling tokenization** (Kudo, 2018)
- **WordPiece** (Schuster and Nakajima, 2012)

All have 2 parts:

- A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
- A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,..., a, b, c, d....}

Repeat:

- Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
- Add a new merged symbol 'AB' to the vocabulary
- Replace every adjacent 'A' 'B' in the corpus with 'AB'.

Until k merges have been done.

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '___' before space in training corpus

Next, separate into letters.

BPE token learner

Original (very fascinating 🤔) corpus:

low low low low low lowest lowest newer newer newer
newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

vocabulary

—, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

Merge **e r** to **er**

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, **er**

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Merge **er _** to **er_**

corpus

5 l o w _
2 l o w e s t _
6 n e w e r_
3 w i d e r_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

Merge **n e** to **ne**

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

BPE

The next merges are:

Merge	Current Vocabulary
(ne, w)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new
(l, o)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo
(lo, w)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low
(new, er—)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low, newer—
(low, —)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low, newer—, low—

BPE token **segmenter** algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test frequencies don't play a role)

So: merge every **e r** to **er**, then merge **er _** to **er_**, etc.

Result:

- Test set "n e w e r _" would be tokenized as a full word
- Test set "l o w e r _" would be two tokens: "low er_"

Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

A **morpheme** is the smallest meaning-bearing unit of a language

- *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*

WordPiece

- **WordPiece** is a **subword-based tokenization algorithm** used in models like **BERT**, originally developed by Google for machine translation.
- It builds a **vocabulary of subword units** and tokenizes text into these units for **efficient representation** of known and unknown words.

WordPiece – Training Phase

1. **Initialize** vocabulary with all individual characters (plus special tokens).
2. Treat each word in the training corpus as a **sequence of characters** plus an end-of-word marker.
E.g., "playing" → ["p", "l", "a", "y", "i", "n", "g", "</w>"]
3. Count frequencies of **all adjacent symbol pairs**.
4. Merge the **most frequent pair** into a new symbol and update all word representations.
5. Repeat steps 3–4 until the vocabulary reaches the desired size (e.g., 30,000 tokens).

This is similar to **Byte-Pair Encoding (BPE)**, but WordPiece selects merges based on **likelihood maximization**, not raw frequency.

Example

Given a toy corpus: Start with character-level tokens
(all words split into characters +
</w> marker):

unrelated

[u n r e l a t e d </w>]

unwanted

[u n w a n t e d </w>]

unhappy

[u n h a p p y </w>]

undo

[u n d o </w>]

Count adjacent pairs:

u n: 4 times

n w: 1

n h: 1

n d: 1

...

Merge most frequent pair: u n $\rightarrow$ un

Update corpus:

[un r e l a t e d </w>]

[un w a n t e d </w>]

[un h a p p y </w>]

[un d o </w>]

Repeat: Merge un h $\rightarrow$ unh, then h a $\rightarrow$ ha, etc.

Eventually you get vocabulary entries like:

un, unh, happy, ed, want, do, ##ed, ##ness, ##ing

What Is Likelihood Maximization in WordPiece?

Let's assume we have a corpus of words and an initial vocabulary of individual characters.

The goal is to **merge subword pairs** in a way that **maximizes the probability of the training corpus** using a unigram language model, where:

Each word is represented as a sequence of subwords from the current vocabulary.

The **probability of a word** is the product of the probabilities of its subwords.

At each step:

Compute the likelihood of the corpus under the current vocabulary.

For every possible pair of symbols, simulate merging it.

Choose the merge that results in the **largest increase in likelihood**.

Mathematical Intuition

Suppose we tokenize word $w = t_1, t_2, \dots, t_k$ using subwords.

$$P(\text{corpus}) = \prod_{t \in \text{tokens}} P(t)$$

with MLE estimated as frequency:

$$P(t) = \frac{\text{count}(t)}{\text{total tokens}}$$

Log-likelihood (LL):

$$LL = \sum_{t \in \text{tokens}} \text{count}(t) \cdot \log P(t)$$

The goal is to choose a merge (say, merge $A + B \rightarrow AB$) that **maximizes** this likelihood.

We do:

For each candidate merge, recompute tokenization of all words.

Compute the updated probabilities of subwords.

Pick the merge that yields the highest increase in total log-likelihood:

$$\Delta \log L = LL_{\text{after}} - LL_{\text{before}}$$

Corpus = ["unhappiness", "unhappy", "happiness"]

Initial **vocabulary** = all characters:

{u, n, h, a, p, y, i, e, s}

Tokenization at start:

"unhappiness" → u n h a p p i n e s s

"unhappy" → u n h a p p y

"happiness" → h a p p i n e s s

Token Count=11+7+9=27

WordPiece models the probability of the corpus as:

$$P(\text{corpus}) = \prod_{t \in \text{tokens}} P(t)$$

with MLE estimated as frequency:

$$P(t) = \frac{\text{count}(t)}{\text{total tokens}}$$

Log-likelihood (LL):

$$LL = \sum_{t \in \text{tokens}} \text{count}(t) \cdot \log P(t)$$

Compute Pair Frequencies

Let's count frequent pairs (adjacent):

$(u,n) \rightarrow 2$ ("unhappiness", "unhappy")

$(h,a) \rightarrow 3$

$(p,p) \rightarrow 3$

$(n,e) \rightarrow 2$

$(e,s) \rightarrow 2$

Candidate Merge — Example (u,n → “un”)

If we merge (u,n):

"unhappiness" → un h a p p i n e s s

"unhappy" → un h a p p y

"happiness" → h a p p i n e s s

New token count = 25 (one less each time "un" occurs).

Likelihood Computation

Before merge:

freq(u)=2, freq(n)=3, freq(h)=3, freq(a)=3, freq(p)=5, freq(y)=1, freq(i)=2,
freq(e)=2, freq(s)=4

total=26

For example:

$P(u)=2/27, P(n)=3/27, P(h)=3/27, \dots$

Log-likelihood contribution:

$$LL = \sum_t c(t) \log \frac{c(t)}{27}$$

After merge ($u, n \rightarrow un$):

$\text{freq}(un)=2, \text{freq}(h)=3, \text{freq}(a)=3, \text{freq}(p)=5, \text{freq}(y)=1,$
 $\text{freq}(i)=2, \text{freq}(e)=2, \text{freq}(s)=4$

total=25

Now:

$P(un)=2/25, P(h)=3/25, \dots$

Compute new:

$$LL' = \sum_t c(t) \log \frac{c(t)}{25}$$

Compare Log-Likelihoods

The merge that **maximizes $LL' - LL$** is chosen.

If merging (u,n) increases LL more than merging (h,a) or (p,p), we add "un" to the vocabulary.

Otherwise, we pick whichever pair improves likelihood the most.

Pair Candidate	Tokens Reduced	Old LL	New LL	ΔLL (improvement)
$(u,n) \rightarrow \text{"un"}$	-2	LL	LL'	$\Delta LL1$
$(h,a) \rightarrow \text{"ha"}$	-3	LL	LL'	$\Delta LL2$
$(p,p) \rightarrow \text{"pp"}$	-3	LL	LL'	$\Delta LL3$

The pair with the **highest ΔLL** is chosen.

Iterative Training

Merge (u,n) $\rightarrow$ "un"

Next, recompute likelihood and merge (h,a) $\rightarrow$ "ha"

Then (p,p) $\rightarrow$ "pp"

Continue until vocabulary size target (e.g., 30,000 tokens) is reached.

BPE and wordPiece Example

Take this corpus of 10 words (k = 10 merges will be applied during training):

Corpus (10 words):

low lower newest wider low lowest newer wider newer
widest

Also assume _ is the word end indicator

Assume we train a character-level BPE and perform 10 merges (k = 10). Come up with all 10 merges.

Find the tokens for following test words "lowest newer widow"

Basic Text Processing

Word Normalization and other issues

Word Normalization

Putting words/tokens in a standard format

- U.S.A. or USA
- uhhuh or uh-huh
- Fed or fed

Case folding

Applications like IR: reduce all letters to lower case

- Since users tend to use lower case
- Possible exception: upper case in mid-sentence?
 - e.g., *General Motors*
 - *Fed* vs. *fed*
 - *SAIL* vs. *sail*

For sentiment analysis, MT, Information extraction

- Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Represent all words as their lemma, their shared root
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')
→ **querer** 'want'
- *He is reading detective stories*
→ *He be read detective story*

Porter Stemmer

Based on a series of rewrite rules run in series

- A cascade, in which output of each pass fed to next pass

Some sample rules:

ATIONAL → ATE (e.g., relational → relate)

ING → ϵ if stem contains vowel (e.g., motoring → motor)

SSES → SS (e.g., grasses → grass)

Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:
- **Uygarlastiramadiklarimizdanmissinizcasina**
- `(behaving) as if you are among those whom we could not civilize’
- **Uygar** `civilized’ + **las** `become’
 - + **tir** `cause’ + **ama** `not able’
 - + **dik** `past’ + **lar** `plural’
 - + **imiz** `p1pl’ + **dan** `abl’
 - + **mis** `past’ + **siniz** `2pl’ + **casina** `as if’

Sentence Segmentation

!, ? mostly unambiguous but **period** “.” is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.

Morphological Analysis

- Human language is a complicated thing.
- We use it to express our thoughts and we receive information and infer its meaning.
- Trying to understand language all together is not a viable approach.
- Linguists have developed whole disciplines that look at language from different perspectives and at different levels of detail.
- The point of morphology, for instance, is to study the variable forms and functions of words
- Words are defined in most languages as the smallest linguistic units that can form a complete utterance by themselves.
- The minimal parts of words that deliver aspects of meaning to them are called morphemes.

Lexemes

- By the term word, we often denote not just the one linguistic form in the given context but also the concept behind the form and the set of alternative forms that can express it. Such sets are called lexemes or lexical items, and they constitute the lexicon of a language.
- Lexemes can be divided by their behaviour into the lexical categories of verbs, nouns, adjectives, conjunctions, particles, or other parts of speech.
- The citation form of a lexeme, by which it is commonly identified, is also called its **lemma**.
- When we convert a word into its other forms, such as turning the singular mouse into the plural mice, we say we **inflect the lexeme**.
- When we transform a lexeme into another one that is morphologically related, regardless of its lexical category, we say we **derive** the lexeme: for instance, the nouns receiver and reception are derived from the verb receive.

Morphemes

- Morphological theories differ on whether and how to associate the properties of word forms with their structural components.
- These components are usually called segments or morphs.
- The morphs that by themselves represent some aspect of the meaning of a word are called morphemes of some function.
- Human languages employ a variety of devices by which morphs and morphemes are combined into word forms.

Morphology

- Morphology is the domain of linguistics that analyses the internal structure of words.
 - Morphological analysis – exploring the structure of words
 - Words are built up of minimal meaningful elements called morphemes: played = play-ed cats = cat-s unfriendly = un-friend-ly
 - Two types of morphemes:
 - I. Stems: play, cat, friend
 - II. Affixes: -ed, -s, un-, -ly
 - Two main types of affixes:
 - I. Prefixes precede the stem: un
 - II. Suffixes follow the stem: -ed, -s, un-, -ly
- Stemming = find the stem by stripping off affixes
- play = play replayed = re-play-ed computerized = comput-er-ize-d

Morphological Analysis in NLP

Purpose: To understand the structure of words and their components.

Key Tasks:

- **Stemming:** Reducing words to their root form (e.g., "running" to "run").
- **Lemmatization:** Converting words to their dictionary form (lemma), considering the context (e.g., "better" to "good").
- **Morpheme Boundary Detection:** Identifying where morphemes begin and end within a word.
- **Morphological Feature Tagging:** Assigning grammatical features (e.g., tense, number, gender) to morphemes.
- **Morphological Reinflection:** Generating different forms of a word based on inflectional rules.

Morphological Typology

- Morphological typology is a way of classifying the languages of the world that groups languages according to their common morphological structures.
- The field organizes languages on the basis of how those languages form words by combining morphemes.
- The morphological typology classifies languages into two broad classes of synthetic languages and analytical languages.
- Analytic languages contain very little inflection, instead relying on features like word order and auxiliary words to convey meaning.
- Synthetic languages, ones that are not analytic, are divided into two categories: agglutinative and fusional languages.
- Agglutinative languages rely primarily on discrete particles (prefixes, suffixes, and infixes) for inflection, ex: inter+national = international, international+ize = internationalize.
- While fusional languages "fuse" inflectional categories together, often allowing one word ending to contain several categories, such that the original root can be difficult to extract (anybody, newspaper).

Morpheme:

Definition: The smallest unit of language that carries meaning.

Types:

- Free morphemes: Can stand alone as a word (e.g., "cat", "run").
- Bound morphemes: Must be attached to other morphemes (e.g., prefixes like "un-" or suffixes like "-ed").

Examples: "unhappy" has two morphemes: "un-" (bound) and "happy" (free).

Lexeme:

Definition: A word and all its inflected forms.

Examples: The lexeme "RUN" includes the forms "run", "runs", "ran", and "running". The lexeme "HOPE" includes "hope", "hopes", "hoped", and "hoping".

Issues and Challenges

Morphology, the study of word structure, faces challenges in handling irregularities, ambiguities, and productivity in language. Here's a more detailed breakdown:

1. Irregularity:

- Many languages have irregular verb forms (e.g., "went," "ran") and noun plurals (e.g., "children") that don't follow standard morphological rules.
- This makes it difficult for computational systems to accurately process and generate these forms.
- Irregularities also exist in other word classes and can be lexically specific.

2. Ambiguity:

- Ambiguity can occur at different levels of morphology. For example, a single morpheme (the smallest meaningful unit of language) can have multiple meanings (e.g., the English plural marker "-s" can indicate possession or plurality).
- Morphological ambiguity can also arise from the way morphemes combine to form words, leading to multiple possible interpretations.
- For example, the word "unlockable" can be interpreted as "not lockable" or "able to be unlocked".

3. Productivity:

- Morphological productivity refers to the ability of a language to create new words. While some word formation processes are highly productive (e.g., adding "-ing" to verbs to create nouns), others are more restricted.
- Computational models need to be able to predict and handle the creation of new words, which can be challenging due to the varying degrees of productivity.
- This is particularly relevant in areas like natural language processing, where systems need to be able to understand and generate novel words.

Models and Techniques for morphological analysis

Finite State Transducers (FST): A common approach for morphological parsing, inputting words and outputting their stems and modifiers.

Indexed Lookup Methods: Using radix trees to analyze word forms, though can be less effective for morphologically complex languages.

Neural Networks: Increasingly used, especially for languages with abundant training data, to learn character-level language models without explicit morphological parsing.